for us automatically. This lets us use the `NewMessage.changeset/1` function on line 3. (An alias at the top makes it more convenient for us to call this function.)

The main test function that you'll use is `assert/1`. This function is small but mighty. You can do simple equality assertions. For example, `assert 1 == 1` is a valid assertion line. But you can also use pattern matching. For example, `assert %{a: _} = %{a: 1}` is a valid assertion.

The power of the pattern assertion style is that ExUnit will show you exactly what diverges when the pattern isn't matched. It's capable of printing this out in an easy-to-read, color-coded format, so you can tell exactly what's different between the left and right sides of the assertion.

All of our tests use pattern assertions to verify that our changeset shape matches an expected value. For example, line 11 asserts that there are no errors in the changeset and that the phone number matches a specific value. Because pattern matching is used, we only need to check for the fields that we care about. (If we were to use equality comparison, then we'd need to exactly match every single field.)

Try changing the assertion pattern so that the test fails. You should see the exact problem highlighted by ExUnit.

These tests are simple. They use functions that don't touch the database or require setup. Let's look at a more complex test case next. You'll see how to test Ecto queries.

What to Test



I don't believe that everything should be tested. There has to be a balance between test value, effort, and business value. That said, I strongly believe in the importance of testing and that you should have a robust test suite for any production application.

Usually, I don't test the framework. In the case of our Ecto schemas, I wouldn't test that our schema has fields or that it operates a certain way. I know these things to be true. I would test that the required fields work correctly and that any validations work correctly because those are implemented by my application.

When in doubt, always test. But don't let testing be the barrier to finishing your feature.